

Implementation and Evaluation of a modular multiplier for cryptographic cores

Alexandra Tsakiri, AEM: 3779

Department of Electrical and Computer Engineering
University of Thessaly
Volos, Greece
Email: altsakiri@uth.gr

Christos Mountakis, AEM: 3801

Department of Electrical and Computer Engineering
University of Thessaly
Volos, Greece
Email: cmountakis@uth.gr

Abstract—This work presents the design, implementation, and evaluation of a modular Montgomery multiplier for cryptographic cores on the Alveo U250 FPGA. Hardware implementation on the FPGA fabric is compared against a software implementation running in python. Multiple operand sizes were considered to analyze execution time.

Index Terms—Montgomery Multiplier, Modular Arithmetic, Cryptography, Alveo U250, FPGA, Software, Timing, Hardware Security.

I. INTRODUCTION

Modular exponentiation is a fundamental operation in public-key cryptography, forming the computational core of algorithms such as RSA, Diffie–Hellman, and elliptic curve cryptography. At the heart of modular exponentiation lies modular multiplication, which must be executed efficiently to meet the stringent performance requirements of modern cryptographic systems.

Montgomery multiplication is a widely adopted technique for accelerating modular arithmetic by eliminating costly division operations [1], [2]. By transforming operands into the Montgomery domain, repeated modular multiplications can be performed using only additions, shifts, and multiplications, making the algorithm particularly well-suited for hardware implementation.

In this work, a Montgomery modular multiplier is designed and implemented in Verilog and synthesized for the Xilinx Alveo U250 FPGA. Two architectural approaches are evaluated: a sequential (generic) implementation and an ultra-parallel implementation. Both designs target large operand sizes relevant to cryptographic applications, with a focus on 1024-bit moduli.

The study analyzes execution time, resource utilization, and architectural trade-offs between the two implementations using Vivado synthesis and implementation results. Particular attention is given to the counterintuitive observation that the ultra-parallel architecture achieves significantly higher performance while exhibiting lower logic utilization, highlighting the impact of modern FPGA synthesis optimizations on highly parallel arithmetic structures.

II. CHALLENGES

The design and evaluation of a high-performance Montgomery multiplier on FPGA presents several technical challenges:

- **Architectural Design Trade-offs:** Selecting an appropriate balance between sequential control logic and parallel arithmetic to optimize performance without excessive resource consumption.
- **Synthesis Optimization Effects:** Understanding and interpreting Vivado’s optimization behavior, particularly the unexpected reduction in logic utilization observed in the ultra-parallel implementation compared to the sequential design.
- **Arithmetic Mapping on FPGA Fabric:** Managing large 64-bit by 64-bit multiplications that exceed the native capabilities of DSP48E2 blocks, leading the synthesis tool to favor LUT-based implementations.
- **Fair Performance Evaluation:** Accurately estimating execution latency based on cycle counts and clock frequency, while restricting independent synthesis and implementation in Vivado to 1024-bit operands, with 512-bit and 2048-bit figures scaled from that result.

III. METHODOLOGY

The methodology followed in this work consists of the following steps:

- 1) Designing and implementing a Montgomery modular multiplier in Verilog, targeting large operand sizes relevant to cryptographic applications.
- 2) Synthesizing and implementing the design on the Xilinx Alveo U250 FPGA using Vivado, for a 1024-bit configuration.
- 3) Performing timing analysis to determine the maximum achievable clock frequency after place-and-route.
- 4) Evaluating architectural variants of the Montgomery multiplier, including sequential and ultra-parallel implementations.
- 5) Extracting latency, throughput, and resource utilization metrics from Vivado reports.
- 6) Analyzing and comparing the performance and resource trade-offs between the examined hardware architectures.

- 7) Comparing the obtained hardware performance results against a reference software implementation to provide an indicative performance baseline.

IV. IMPLEMENTATION

A. Code

The Montgomery multiplier was designed from scratch and implemented in Verilog specifically for this work. The design was developed to target FPGA execution on the Alveo U250 platform and to support multiple architectural variants. The implementation was iteratively refined and optimized to achieve correct functionality, high performance, and efficient resource utilization.

Two main versions of the Montgomery multiplier were developed:

- a generic (sequential) implementation, and
- an ultra-parallel implementation optimized for throughput.

Both versions operate on 1024-bit operands and internally decompose the computation into smaller word-level multiplications.

B. Generic (Sequential) Implementation

The generic implementation follows an operand decomposition strategy, splitting the 1024-bit operands into sixteen 64-bit words. However, unlike the ultra-parallel version, these word-level multiplications are executed sequentially over multiple clock cycles.

Although, in theory, a single multiplier could be reused for all operations, this approach is not practical due to the complexity of the control logic and timing constraints. As a result, the generic design still instantiates multiple multiplication units, but schedules their operation sequentially. This leads to a significantly higher cycle count per Montgomery multiplication compared to the ultra-parallel implementation.

C. Ultra-Parallel Implementation

The ultra-parallel implementation operates on 64-bit words, computing one complete 64-bit word per clock cycle, with sixteen word-level multiplications executed in parallel for 1024-bit operands.

As a result, the ultra-parallel architecture performs a total of sixteen word-level multiplications simultaneously, significantly reducing the total number of cycles required to complete a Montgomery multiplication. This design emphasizes data-path parallelism and minimizes sequential control dependencies, allowing the synthesis tool to aggressively optimize the arithmetic logic.

D. Interface Optimization and Resource Constraints

During synthesis and implementation, the initial design exhibited excessive resource utilization and could not be successfully placed and routed on the Alveo U250 FPGA. The primary limiting factor was the large number of input and output ports, which resulted in high I/O utilization.

To address this issue, the design was modified by parameterizing the input and output interfaces. Instead of exposing wide parallel data buses, operands and results are transferred using parameterized interfaces, significantly reducing I/O pressure. In a real-world application, these I/Os would typically be transferred via standardized buses, such as AXI4 interfaces, driven by a host CPU to the FPGA accelerator. This optimization enabled successful implementation on the target FPGA while preserving the internal computational architecture.

V. HARDWARE OPERATION AND TIMING ANALYSIS

A. Montgomery Modular Exponentiation in Hardware

The proposed hardware accelerator computes modular exponentiation using the square-and-multiply algorithm entirely in the Montgomery domain. The operands c , d , and n are 1024-bit integers, with n assumed to be odd. All modular operations are performed using a dedicated Montgomery multiplier, which serves as the fundamental computational unit of the design.

Prior to exponentiation, the base operand c is converted into Montgomery representation using a single Montgomery multiplication. The result register is initialized to the Montgomery representation of unity. The exponentiation process iterates over all bits of the exponent d , starting from the most significant bit to the least significant bit.

At each iteration, the hardware performs one mandatory Montgomery multiplication corresponding to the squaring operation. If the current exponent bit is 1, an additional Montgomery multiplication is executed to multiply the intermediate result by the base operand. All intermediate values remain in Montgomery form throughout the computation, eliminating the need for explicit modular reductions.

After processing all exponent bits, a final Montgomery multiplication converts the result back to the standard integer representation. Consequently, the total execution time is determined solely by the number of Montgomery multiplications and the latency of each multiplication.

B. Operation Count Analysis

For a 1024-bit exponent, the total number of Montgomery multiplications can be derived analytically. The contributions for best-case, average-case, and worst-case scenarios are summarized in Table I.

TABLE I
MONTGOMERY MULTIPLICATION COUNT FOR 1024-BIT
EXPONENTIATION

Operation	Best	Average	Worst
Convert base to Montgomery	1	1	1
Squares (all bits)	1024	1024	1024
Conditional multiplies	1	512	1024
Final conversion	1	1	1
Total MontMul	1027	1538	2050

C. Cycle-Level Timing Analysis

The latency of a single Montgomery multiplication depends on the hardware architecture. Two implementations were evaluated:

- **Sequential (Generic):** 274 cycles per Montgomery multiplication
- **Ultra-Parallel:** 19 cycles per Montgomery multiplication

Both architectures were synthesized and implemented on the Alveo U250 FPGA, achieving a post-implementation clock period of 4.5 ns (222 MHz). The resulting execution time per Montgomery multiplication is shown in Table II.

TABLE II
LATENCY PER MONTGOMERY MULTIPLICATION

Implementation	Cycles	Time per MontMul
Generic (Sequential)	274	1.233 μ s
Ultra-Parallel	19	85.5 ns

D. CPU vs FPGA Comparison

To evaluate hardware acceleration, the FPGA results are compared with CPU execution times measured on a general-purpose processor. Each CPU value corresponds to the mean execution time over 50 runs. The comparison and achieved speedup are summarized in Table III.

TABLE III
CPU VS FPGA PERFORMANCE FOR 1024-BIT MODULAR EXPONENTIATION

Case	CPU (μ s)	FPGA Ultra (μ s)	FPGA Seq (μ s)	Speedup (Ultra)
Best	3699.7	87.8	1267.0	42.1 $\times$
Average	4558.9	131.5	1896.0	34.7 $\times$
Worst	4537.7	175.3	2527.0	25.9 $\times$

Key Observations:

- The ultra-parallel FPGA implementation achieves up to 42.1 $\times$ speedup over the CPU.
- The sequential FPGA implementation is faster than the CPU for all cases.
- CPU execution time shows limited variation (1.23 $\times$) between best and worst cases, indicating data-dependent behavior. The average case (4558.9 μ s) measuring slightly above the worst case (4537.7 μ s) is attributed to CPU measurement variance rather than a modeling error.
- FPGA execution exhibits approximately 2 $\times$ variation between best and worst cases, reflecting the dependence on exponent bit density.
- The observed speedup is primarily due to architectural parallelism rather than clock frequency scaling.

VI. RESOURCE UTILIZATION ANALYSIS

A. Why the Ultra-Parallel Version Uses Less Resources

The difference in resource utilization is due to how Vivado optimizes multiplications:

a) Sequential (Generic)::

- Multiplications are executed sequentially over 274 cycles.
- Each multiplication must be preserved in logic due to state machine dependencies.
- Optimization is limited because multiplication logic is interleaved with control flow.

b) Ultra-Parallel::

- All 16 multiplications are performed in a single cycle.
- Vivado interprets the operations as pure combinational arithmetic.
- Modern synthesis tools efficiently optimize repeated patterns (`ui * b_reg[k*WORD +: WORD]`) using LUTs.
- Overall logic usage is reduced despite performing more operations per cycle.

B. Why No DSPs Are Used

Both designs report DSP usage of 0:

- 64-bit $\times$ 64-bit multiplications exceed a single DSP48E2 block capability (27 $\times$ 18 bits).
- Implementing 64 $\times$ 64 multiplication would require multiple DSPs in a tree, which is inefficient.
- LUT-based multiplication provides better timing and resource trade-offs, especially for sparse or large operands.

This explains why the ultra-parallel architecture achieves lower resource usage while avoiding DSPs.

VII. CPU BENCHMARK AND PERFORMANCE COMPARISON FOR 512-BIT MODULAR EXPONENTIATION

A. CPU Benchmark

To establish a baseline, modular exponentiation was executed on a standard CPU for 512-bit operands using Python's built-in `pow(base, exponent, modulus)` function. Three representative cases were considered: best-case, average-case, and worst-case, depending on the number of ones in the exponent. The estimated Montgomery operations and measured CPU performance are summarized in Table IV.

TABLE IV
CPU BENCHMARK FOR 512-BIT MODULAR EXPONENTIATION (50 RUNS PER CASE)

Case	Exponent Ones	MontOps	Mean (us)	Min (us)	Max (us)
Best	1	515	656.9	625.3	719.5
Average	256	770	793.8	741.6	879.6
Worst	512	1026	787.0	730.1	1098.9

Observations:

- CPU execution time increases with the number of ones in the exponent, except that the average case (793.8 μ s) measures slightly above the worst case (787.0 μ s), attributed to CPU measurement variance.
- Variation across cases is approximately 1.20 $\times$ (best $\rightarrow$ worst).

B. CPU vs FPGA Performance Comparison

The FPGA-U and FPGA-S figures below are scaled from the synthesized 1024-bit design's per-MontMul latency (85.5 ns and 1.233 μ s respectively); 512-bit and 2048-bit variants were not independently synthesized and placed on the Alveo U250.

The CPU results were compared against the FPGA implementations on the Alveo U250, including ultra-parallel and sequential architectures. The total execution times and corresponding speedups are summarized in Table V.

TABLE V
CPU VS FPGA PERFORMANCE FOR 512-BIT MODULAR EXPONENTIATION

Case	MontOps	CPU(us)	FPGA-U(us)	FPGA-S(us)	Speedup
Best	515	657	44.0	635	14.9 $\times$
Average	770	794	65.8	949	12.1 $\times$
Worst	1026	787	87.7	1265	9.0 $\times$

Key Findings:

- The FPGA ultra-parallel implementation achieves a **12.1 $\times$ speedup over CPU** in the average case.
- Sequential FPGA implementation is generally slower than the CPU for this operand size, except in the best-case scenario.
- CPU execution time shows approximately 1.20 $\times$ variation between best and worst cases.
- FPGA shows approximately 2.0 $\times$ variation, reflecting dependency on exponent bit density.
- Significant acceleration is achieved due to cycle-level parallelism of the ultra-parallel architecture.

VIII. CPU BENCHMARK AND PERFORMANCE COMPARISON FOR 2048-BIT MODULAR EXPONENTIATION

A. CPU Benchmark

The CPU was used to measure execution time of modular exponentiation for 2048-bit operands, computed using Python's built-in `pow(base, exponent, modulus)` function. Three cases were evaluated: best-case, average-case, and worst-case, based on exponent bit density. The number of Montgomery operations and CPU performance are shown in Table VI.

TABLE VI
CPU BENCHMARK FOR 2048-BIT MODULAR EXPONENTIATION (20 RUNS PER CASE)

Case	Exponent Ones	MontOps	Mean (us)	Min (us)	Max (us)
Best	1	2051	25891.4	24531.0	27485.5
Average	1024	3074	31766.0	30972.7	33745.3
Worst	2048	4098	32431.0	30727.7	41203.8

Observations:

- CPU execution time scales with the number of ones in the exponent.
- Variation across cases is approximately 1.25 $\times$ (best $\rightarrow$ worst).

B. CPU vs FPGA Performance Comparison

As with the 512-bit case, the FPGA-U and FPGA-S figures below are scaled from the synthesized 1024-bit design's per-MontMul latency rather than from an independently synthesized 2048-bit implementation.

The CPU results were compared to FPGA implementations (Alveo U250), including ultra-parallel and sequential architectures. Table VII summarizes total execution times and speedups.

TABLE VII
CPU VS FPGA PERFORMANCE FOR 2048-BIT MODULAR EXPONENTIATION

Case	MontOps	CPU(us)	FPGA-U(us)	FPGA-S(us)	Speedup
Best	2051	25891	175.4	2529	147.6 $\times$
Average	3074	31766	262.8	3790	120.9 $\times$
Worst	4098	32431	350.4	5053	92.6 $\times$

Key Findings:

- Ultra-parallel FPGA implementation achieves **$\sim 121\times$ speedup over CPU** in the average case.
- Sequential FPGA implementation is significantly faster than the CPU for large operands.
- CPU execution shows approximately 1.25 $\times$ variation between best and worst cases.
- FPGA shows approximately 2.0 $\times$ variation, reflecting dependence on exponent bit density.
- Ultra-parallel architecture provides massive acceleration due to hardware-level parallelism.

IX. POWER

The power consumption of the proposed FPGA implementations is obtained from the Vivado power analysis reports shown in Fig. 1. The figures present the estimated on-chip power and resource utilization for both the generic sequential and the ultra-parallel architectures. Notably, both designs consume approximately 2.95 W on-chip. For reference, modern general-purpose CPUs typically have a rated TDP in the tens to low hundreds of watts under load; CPU power was not directly measured.

X. CONCLUSION

This work evaluated a Montgomery modular multiplier on the Xilinx Alveo U250 FPGA. The ultra-parallel design achieved significant performance gains over both the sequential FPGA and a CPU reference, reaching speedups up to 42.1 $\times$ (1024-bit) and 147.6 $\times$ (2048-bit). Results demonstrate that architectural parallelism enables substantial acceleration of cryptographic workloads on FPGA, maintaining moderate resource and power consumption, confirming the effectiveness of hardware-accelerated Montgomery multiplication.

REFERENCES

- [1] Wikipedia contributors, "Montgomery modular multiplication," *Wikipedia*, 2025. [Online]. Available: https://en.wikipedia.org/wiki/Montgomery_modular_multiplication. [Accessed: Dec. 29, 2025].
- [2] N. Thampi and M. E. Jose, "Montgomery Multiplier for Faster Cryptosystems," *Procedia Technology*, vol. 25, pp. 392–398, 2016.

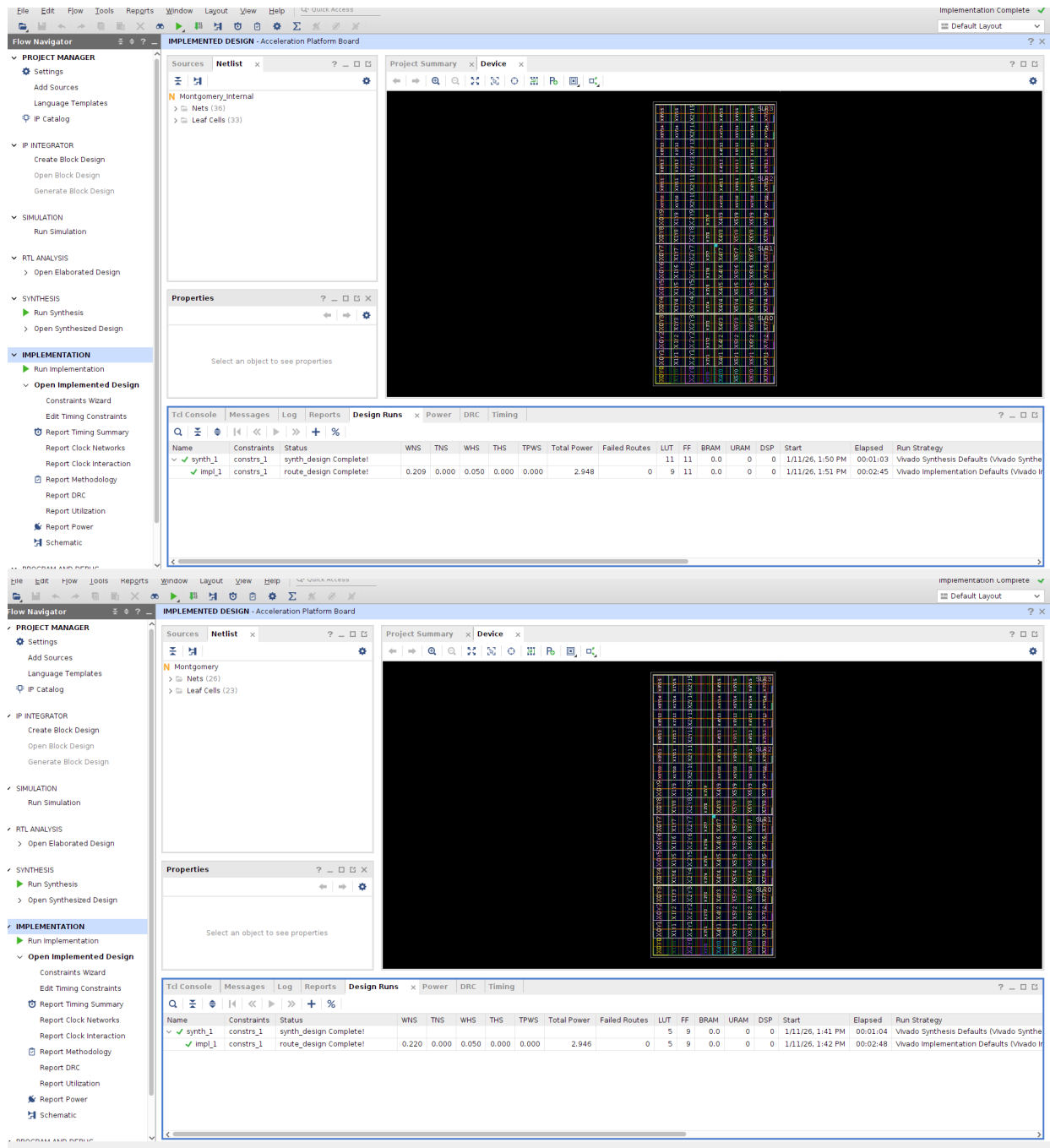


Fig. 1. Vivado power and utilization reports for the 1024-bit modular exponentiation accelerator on Alveo U250: top: generic sequential, bottom: ultra-parallel implementation.